

Bigtable in GCP

Google Cloud Bigtable is a fully managed, scalable NoSQL database service designed for large analytical and operational workloads. It is modeled after Google's internal Bigtable system, which has been used internally by products such as Search, Maps, and Gmail.

Key Features of Bigtable

1. **Wide-column NoSQL Database:** Bigtable is a wide-column store, meaning data is stored in rows and columns, but unlike traditional relational databases, it does not require a fixed schema.
2. **High Scalability:** It can handle massive volumes of data—up to petabytes—across thousands of machines seamlessly.
3. **Low Latency:** Designed for low-latency read and write operations, making it suitable for both real-time and batch workloads.
4. **Fully Managed:** Google Cloud handles infrastructure, patching, replication, and scaling, reducing the need for operational overhead.
5. **Seamless Integration:** Works well with the Google Cloud ecosystem, especially with tools like Dataflow, Dataproc, BigQuery, and Apache HBase clients.

Use Cases

1. **Time Series Data:**
 - Monitoring application performance
 - IoT telemetry data
 - Financial data feeds
2. **Real-time Analytics:**
 - Personalization engines
 - Recommendation systems
 - Real-time dashboards
3. **Machine Learning Pipelines:**
 - Storing features used for training and inference
 - Input for TensorFlow models
4. **Graph and Identity Management:**
 - Storing social network graphs
 - Fraud detection systems

5. High Throughput Applications:

- Serving layer for search engines
- Online transaction systems needing low-latency access

Benefits

1. **Scalability:** Grows with your data needs—scale horizontally with no downtime.
2. **Performance:** Consistently low latency for high throughput applications.
3. **Reliability:** Offers multi-cluster replication for high availability and disaster recovery.
4. **Cost Efficiency:** You pay for what you use—separate pricing for storage and compute allows cost control.
5. **Compatibility:** Supports the HBase API, enabling migration or integration with existing Hadoop ecosystems.
6. **Security:** Integrated with Google Cloud IAM, audit logging, and encryption at rest and in transit.

Limitations

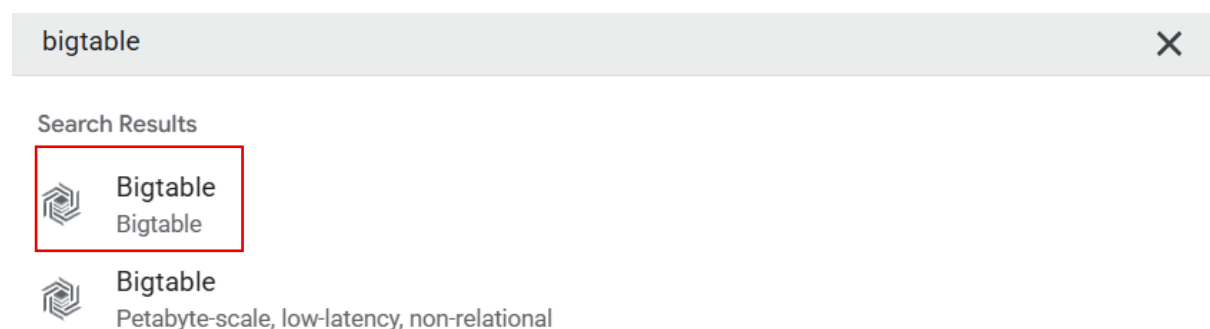
- Not suitable for complex relational queries or multi-row transactions.
- Schema design and row key selection are critical for performance and scalability.
- Better for high-volume workloads rather than low-usage, ad-hoc querying.

Conclusion

Google Cloud Bigtable is a powerful NoSQL solution built for massive scale and speed. It is best suited for applications that require real-time analytics on large datasets, such as telemetry systems, recommendation engines, and machine learning pipelines. Proper use of its design patterns and integrations ensures efficient and reliable performance at scale.

To begin with the Lab

1. In your Google Cloud Console, search for Bigtable and choose the service accordingly.



2. Then, from the dashboard of BigTable, click on Create Instance.



Instances

[+ CREATE INSTANCE](#)[VIEW FLEET HEALTH](#)

Bigtable is a fully managed, wide-column NoSQL database that offers low latency and replication for high availability. To use Bigtable, create an instance and then set up your development environment to access Bigtable so that you can add data and monitor performance. [Learn more](#)

Filter Filter instances

☐ ☒ **Instance ID** Instance name Zones Nodes Storage utilization

No instances to display

3. First, you are going to name your instance. Click on Continue.

A Bigtable instance is a container for your clusters. [Learn more](#)

1 Name your instance

Instance name *
first-bt

For display purposes only

Instance ID *
first-bt

ID is permanent

[CONTINUE](#)

2 Select your storage type

3 Configure your first cluster

[SHOW ADVANCED OPTIONS](#)

[CREATE](#) [CANCEL](#)

Summary



Pricing estimate

\$468 per month

That's about \$0.65 an hour with 0 GB stored.

[SHOW DETAILS](#)

4. For the storage type chose SSD and click on Continue.

A Bigtable instance is a container for your clusters. [Learn more](#)

1 Name your instance

2 Select your storage type

Choice is permanent. Applies to all clusters. Affects cost.

☒ SSD

Lower latency and more rows read per second. Typically used for real-time serving use cases, such as ad serving and mobile app recommendations.

☐ HDD

Higher latency for random reads. Good performance on scans and typically used for batch analytics, such as machine learning or data mining.

CONTINUE

3 Configure your first cluster

SHOW ADVANCED OPTIONS

CREATE

CANCEL

Summary



Pricing estimate

\$468 per month

That's about \$0.65 an hour with 0 GB stored.

SHOW DETAILS

5. In the end, to configure your cluster, just choose a region of your choice. Also you can notice that this service is quite expensive.

6. Then just click on create.

3 Configure your first cluster

A cluster handles application requests for an instance. It contains nodes which determine your cluster's performance and storage limit.

Additional clusters can be added at any time.

Select a cluster ID

ID is permanent

Cluster ID *
first-bt-c1

Select a location

Choice is permanent. Determines where cluster data is stored. To reduce latency and increase throughput, store your data near the services that need it. [Learn more](#)

Region *
asia-south1 (Mumbai)

Zone *
Any

Configure Nodes

Nodes are compute resources that Bigtable uses to manage your data and perform maintenance tasks. Adding nodes helps a cluster handle larger workloads.

☐ Enable 2x node scaling

Choice is permanent. Select to improve cluster performance by reducing boundaries between nodes. If selected, clusters will scale in increments of 2 nodes. This decision will only affect this cluster; clusters in an instance can have different node scaling factors.

Summary



Pricing estimate

\$539 per month

That's about \$0.75 an hour with 0 GB stored.

SHOW DETAILS

7. Here you can see that our Bigtable is ready. Click on view its properties.



Instances

[+ CREATE INSTANCE](#)[VIEW FLEET HEALTH](#)

Bigtable is a fully managed, wide-column NoSQL database that offers low latency and replication for high availability. You can provision Bigtable instances for your workload, then use one of the Bigtable client libraries to develop applications. [Learn more](#)

i You've got a new instance! Connect to it with the cbt command-line tool and create a table. [Learn more](#)

Filter Filter instances

☐	☒	Instance ID	Instance name ↑	Zones	Nodes ?	Storage utilization ?
☐	☒	first-bt	first-bt	asia-south1-a Add	1	i - / -

8. As our instance has not tables right now, so we will create a table in it.

Bigtable

Instance

Overview

Tables

Bigtable Studio

Backups

Application profiles

Observability

System insights

Key Visualizer

Release Notes

All instances > INSTANCE first-bt: Tables

[EDIT INSTANCE](#) [DELETE INSTANCE](#) [SHOW INFO PANEL](#)

Tables [+ CREATE TABLE](#)

Bigtable stores data in massively scalable tables, each of which is a sorted key/value map. A table is composed of rows, each of which typically describes a single entity, and columns, which contain individual values for each row.

Filter Filter tables

☐	Table ID ↑	Cluster	Status	Storage utilization	Automated backup	Latest backup	Change stream ?
--------------------------	------------	---------	--------	---------------------	------------------	---------------	-----------------



This instance has no tables

9. First, we are going to give a name to our table then scroll down.

Name your table

Choice is permanent

Table ID
orders

10. Now we will add our first column family which is named as omd (order metadata).

Add column families (optional)

A Bigtable column family and column qualifier are used in combination to identify a column when reading, writing, or deleting data.

 **New column family** 

Name your column family
Choice is permanent

Column family name *

Set garbage collection policy ?
Garbage collection is a built-in, asynchronous background process

☒ Never collect garbage

☐ Age based policy

☐ Version based policy

☐ Custom policy

DONE

11. And the second column family would be oi (order items). Click on create.

^ New column family

Name your column family

Choice is permanent

Column family name *
oi

Set garbage collection policy

Garbage collection is a built-in, asynchronous background process

☒ Never collect garbage

☐ Age based policy

☐ Version based policy

☐ Custom policy

DONE

12. Here you can see that our table has been created. Now we are going to work with the table, and for that we are going to use a command-line tool called CBT.
13. Now open your Cloud shell and in there if you type cbt you will see that the cloud shell is familiar with this tool.
14. Now we need to configure the cbt tool to be able to work with our instance. For that you need to run the below command. Just change the project ID and instance name.
15. So, when you run the below command, a configuration file is created containing the configuration required by cbt tool.

echo project = PROJECT_ID >> ~/.cbtrc && echo instance = readit-bt >> ~/.cbtrc

16. Now if you run the cat command with the file in the cloud shell then you can see the configuration file content.

```
lordstar697@cloudshell:~ (still-kit-459403-e2)$ cat ~/.cbtrc
project = still-kit-459403-e2
instance = first-bt
```

17. Now we are going to work with the tables, but first we will list our table here in the Cloud Shell.

```
lordstar697@cloudshell:~ (still-kit-459403-e2)$ cbt ls
2025/05/27 10:59:48 -creds flag unset, will use gcloud credential
orders
```

18. Using the below command, we have listed the column family from our table on the terminal.

```
lordstar697@cloudshell:~ (still-kit-459403-e2)$ cbt ls orders
2025/05/27 11:00:53 -creds flag unset, will use gcloud credential
Family Name      GC Policy      Value Type
-----
oi               <never>       {}
omd             <never>       {}
```

19. Now we are going to insert some data into our table for that we run the command given below.

cbt set orders order#17 omd:user=bobj

20. Then we are going to read the contents from our orders table using the read command shown in the snapshot.

```
lordstar697@cloudshell:~ (still-kit-459403-e2)$ cbt read orders
2025/05/27 11:03:36 -creds flag unset, will use gcloud credential
-----
order#17
  omd:user                               @ 2025/05/27-11:03:07.280000
  "bobj"
```

21. In the end, we have counted the row in our order table and then deleted it.

```
lordstar697@cloudshell:~ (still-kit-459403-e2)$ cbt count orders
2025/05/27 11:04:50 -creds flag unset, will use gcloud credential
1
lordstar697@cloudshell:~ (still-kit-459403-e2)$ cbt deleterow orders order#17
2025/05/27 11:04:57 -creds flag unset, will use gcloud credential
```

22. Once you done with the just delete the resources.